

Array:

Collection of similar data types in sequential continuous memory blocks .

Array can be 1D or 2D.

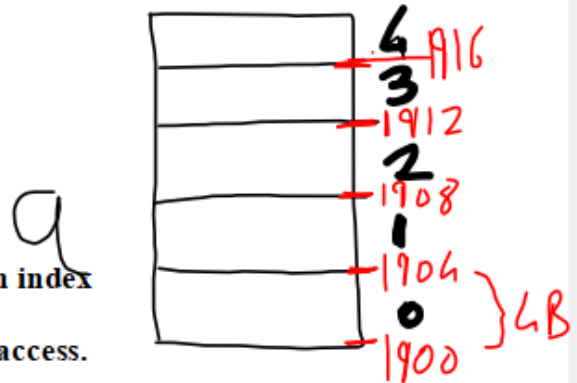
1D: Data is arranged in a sequential manner, one after another, and can be accessible using an index.

Syntax: <data type> <variable> [<size>];
size * datatype memory allocation
int a[5] ----> $5 * 4 = 20$ bytes

An array can be accessed on the basis of an index and

it is possible to subset element-by-element access.

a[index] = a[0] --- 1900 ----> data



Can you explain why the index starts from 0?

a[index] ----> start + (index * size of each block) for address

a[2] ----> $1900 + (2 * 4) = 1900 + 8 = 1908$

a[0] ----> $1900 + 0 = 1900$

int a[]; #not allowed unless static data given

int a[] = {11, 22, 33, 44, 55}; // In this we are not specifying size but we are specifying the number of elements. Size would be taken automatically by the compiler.

// Once created you cannot modify its size.

// Can you create an array of size n?

With C++ yes, by smartly declaring data as a different type

In C++ we support dynamic declaration of data, which means that when and where data is needed, you can declare the data and start using it.

```
int n;
```

```
cout << "Enter n:";
```

```
cin >> n;
```

```
int a[n];
```

C: old style

```
int a[100], n;
```

```
printf("Enter size:", *n);
```

```
for (i=0; i<=n; i++)
```

a[i] read ----> Even if we created an array of size 100, we will only take n elements, leaving 100 - n elements wasted.

C:old style

```
int a[100],n;
```

```
printf("Enter size:","*n);
```

```
for (i=0;i<=n;i++)
```

a[i] read----->Even if we created an array of size 100, we will only take n elements, leaving 100 - n elements wasted.

Arrays are sequentially accessed most of the time using a `for` loop with the index.

```
for(int index=0;index<n;index++)//index 0 to size-1
```

```
{  
    cout<<"\n enter data at ["<<index<<"]:";
```

```
    cin>>a[index];
```

```
} <Type Text Here>
```

$n=5$

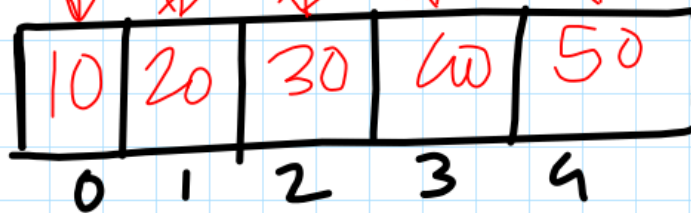
```
for(int index=0;index<n;index++)//index 0 to size-1
```

```
{  
    cout<<"\n enter data at ["<<index<<"]:";
```

```
    cin>>a[index];
```

```
} <Type Text Here>
```

a



2D array--->Is arranged in rows and columns and will require two indexes to access:

1. row index
2. column index

`int m[3][4];`//We always start with row and then column universally.

	0	1	2	3
0	00	01	02	03
1	10	11	12	13
2	20	21	22	23

```
for(int r=0;r<rows;r++)  
{  
    for(int c=0;c<columns;c++)  
    {  
        cout<<"Enter data for m ["<<r<<"]["<<c<<"]:";  
        cin>>m[r][c];  
    }  
}
```

0 0
0 1
0 2
0 3

String:
c style
|
c++ style

c:
`char name[]="amar";`//In traditional C style a string was nothing but a character array.

c++:
In C++ we started a built-in data type derived from character called 'string'.
`string name="amar";`
A string can take any character from the character set of C++.
One of the most important things to learn in programming, because by default the input given by the user is always a string.

String in C++ is indexable, substrings and breaking are possible, and manipulation of string is possible with methods also.

`string name="amar"` →

a	m	a	r
0	1	2	3

 → name
[0]
[1]
[2]
[3]

Normal 'cin' will not work with string. It requires 'getline' to take multiple words.

```
string name;  
cin>>name;//amar panchal  
name<<"amar"
```

This happens because of the concept of tokenization: a token is denoted by a blank. Here in the input we give two tokens, Amar and Panchal, but as seen it will only process one token at a time.

```
getline(cin,variable);
```

//Will allow us to read data till Enter is hit.

```

File Edit Search Run Compile Debug Project Options Window Help
[ ] POINTER_.CPP 1=1
#include<iostream.h>
#include<conio.h>
int main()
{
    int *p;
    int a=10;
    clrscr();
    cout<<"data:"<<*p<<"\taddress"<<p;
    cout<<"\ndata of a:"<<a<<"\t address of a:"<<&a;
    p=&a;
    cout<<"\ndata pointer by p:"<<*p<<"\taddress pointed by p:"<<p;
    a=a+10;
    cout<<"\ndata pointer by p:"<<*p<<"\taddress pointed by p:"<<p;
    *p=*p*2;
    cout<<"\ndata of a:"<<a<<"\t address of a:"<<&a;
    getch();
}
1:1
F1 Help Alt-F8 Next Msg Alt-F7 Prev Msg Alt-F9 Compile F9 Make F10 Menu

```

Pointer:

It is a special mechanism for accessing a direct memory location.

A pointer never points to data. It points to the memory location where data is.

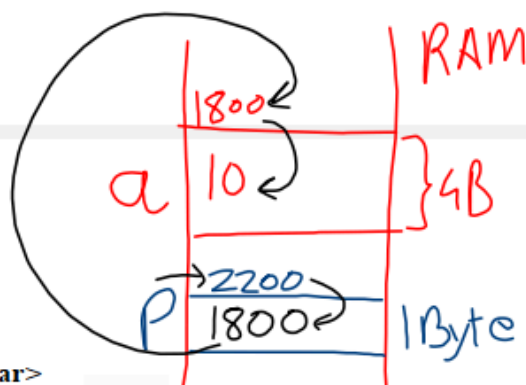
A pointer follows one specific rule: it can only refer to its own type.

That means if a pointer is of integer type then only it can refer to `int`. If it is of float type then only it can refer to `float`. This is because when a pointer refers to a memory, it refers to the block of memory, which is the amount of data it has to read or refer to.

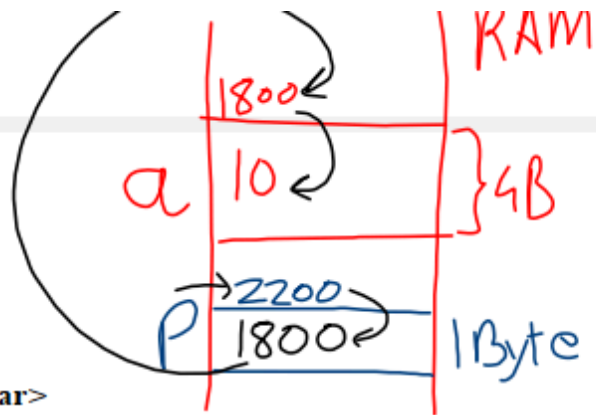
Uses two major operators: * and &

*: Value at address operator

&: Address of operator



```
int a=10;
int *p; // Is created starting with *<var>
```



```
int a=10;
int *p; // Is created starting with *<var>
// This time the pointer is called the garbage pointer as it has some garbage data.
```

```
p=&a; // Assigning the address of 'a' to pointer 'p'. Now 'p' refers to 'a's address.
```

```
a->10
*p->10
&a->1800
p---->1800
```

Changes in **a** will be seen by **p** and changes done by **p** will be seen on **a**.

```
a=a+5;
*p-->15
*p=*p+10
a->25
```

pointer to pointer:

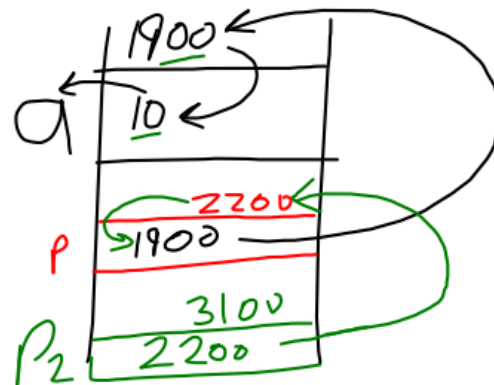
Special pointing mechanism, which refers to a pointer that is referring to data

```
int a=10; ✓
int *p=&a; ✓
int **p2=&p;
```

```
a---->10
**p2--->10
*p---->10
```

```
&a---->1900
p---->1900
p2---->2200
```

```
*p2---->1900
```



Pointer to array:

A pointer, by default, will point to the 0th element of an array and remaining elements can be accessed using pointer arithmetic.

```
int a[]={10,20,30,40,50};
```

```
int *p;
```

```
p=&a; or p=&a[0]; //Means exactly the same
```

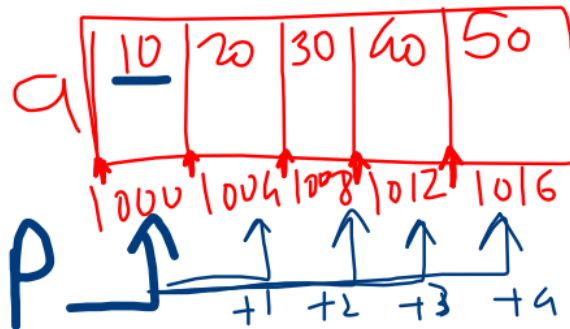
```
*p---->10
```

```
*(p+2)-->30
```

```
*p+2---->12
```

```
*p++---->11
```

```
*(p++)--->20
```



Memory Allocation

Static memory allocation versus dynamic memory allocation.

In static memory

- allocation allocation is done at compile time.

- It is fixed

- easy to get

- Not memory efficient. We can have underflow (less than what we need) or overflow (more than what we need). It is because memory was allocated on prediction not on need.

ex: Buffet lunch or dinner

```
int a[100];
```

In Dynamic memory

- allocation allocation is done at run time.

- It is on demand and variable.

- needs command to give (complex)

- memory efficient. Memory is assigned on a demand basis and can be freed if not needed.

ex: Home food or having food at home

```
int *p=new int[100];
```

```
delete p;
```

```
*p=new int[10];
```